

How to build linear features

Master Degree in Control Systems Engineering 2024-2025
University of Padova

Teachers: Prof. Ruggero Carli & Prof. Gian Antonio Susto

Today's lecture

- How to build linear features (Coarse coding; Tile coding; Radial basis functions)
- Control with function approximation

Building linear features

Linear learning methods are of great interest :

- convergence properties
- very efficient in terms of both data and computation

Building linear features

Linear learning methods are of great interest :

- convergence properties
- very efficient in terms of both data and computation

...but the overall goodness depends on how the states are represented in terms of the features...

Building linear features

Linear learning methods are of great interest :

- convergence properties
- very efficient in terms of both data and computation

...but the overall goodness depends on how the states are represented in terms of the features...

Two observations:

- Choosing features appropriate to the task is an important way of *adding prior domain knowledge* to reinforcement learning systems.

Building linear features

Linear learning methods are of great interest :

- convergence properties
- very efficient in terms of both data and computation

...but the overall goodness depends on how the states are represented in terms of the features...

Two observations:

- Choosing features appropriate to the task is an important way of *adding prior domain knowledge* to reinforcement learning systems.
- Intuitively, the features should correspond to the natural features of the task, those along which generalization is most appropriate.

Building linear features

Linear learning methods are of great interest :

- convergence properties
- very efficient in terms of both data and computation

...but the overall goodness depends on how the states are represented in terms of the features...

Two observations:

- Choosing features appropriate to the task is an important way of *adding prior domain knowledge* to reinforcement learning systems.
- Intuitively, the features should correspond to the natural features of the task, those along which generalization is most appropriate.

valuing geometric objects  features for each possible shape, color, size

Building linear features

Linear learning methods are of great interest :

- convergence properties
- very efficient in terms of both data and computation

...but the overall goodness depends on how the states are represented in terms of the features...

Two observations:

- Choosing features appropriate to the task is an important way of *adding prior domain knowledge* to reinforcement learning systems.
- Intuitively, the features should correspond to the natural features of the task, those along which generalization is most appropriate.

valuing geometric objects  features for each possible shape, color, size

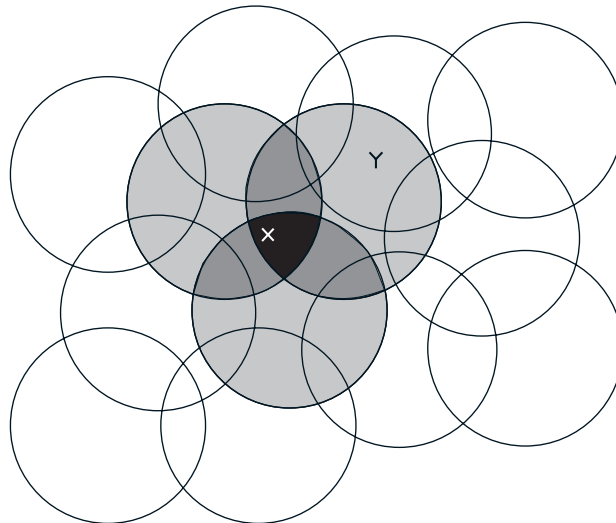
valuing states of a mobile robot  features for locations, degrees of remaining battery power, recent sonar readings

Today's lecture

- How to build linear features (**Coarse coding**; Tile coding; Radial basis functions)
- Control with function approximation

Coarse coding

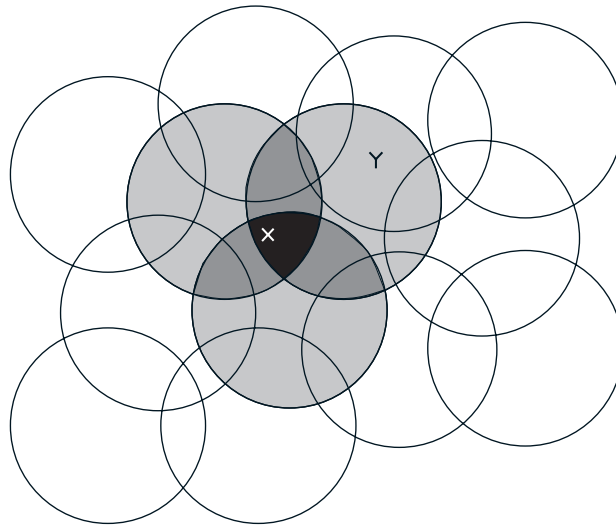
- Consider a task in which the state set is continuous and two-dimensional.
- A state in this case is a point in 2-space, a vector with two real components.
- **Features** $\longrightarrow$ **circles** in state space



Coarse coding – Bynary features

If the state is INSIDE a circle $\rightarrow$ the corresponding feature has the *value 1*
(it is said to be *present*)

If the state is NOT INSIDE a circle $\rightarrow$ the corresponding feature has the
value 0 (it is said to be *absent*)

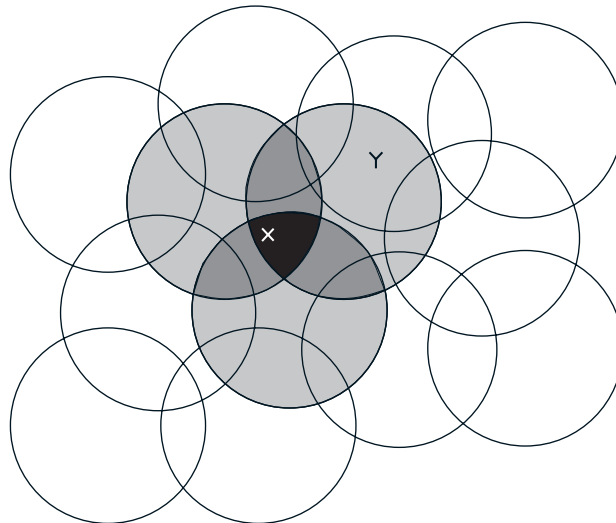


Coarse coding – Bynary features

If the state is INSIDE a circle $\rightarrow$ the corresponding feature has the *value 1*
(it is said to be *present*)

If the state is NOT INSIDE a circle $\rightarrow$ the corresponding feature has the
value 0 (it is said to be *absent*)

This kind of 1–0-valued feature is called a **binary feature**

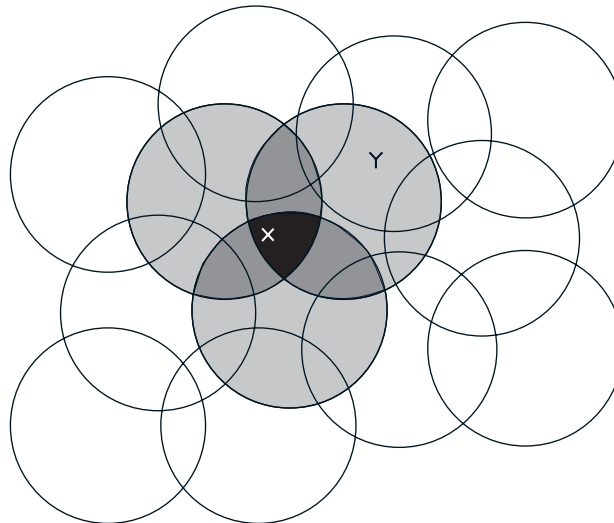


Coarse coding – Bynary features

If the state is INSIDE a circle → the corresponding feature has the *value 1*
(it is said to be *present*)

If the state is NOT INSIDE a circle → the corresponding feature has the
value 0 (it is said to be *absent*)

*Given a state, which binary features are present indicate within
which circles the state lies*

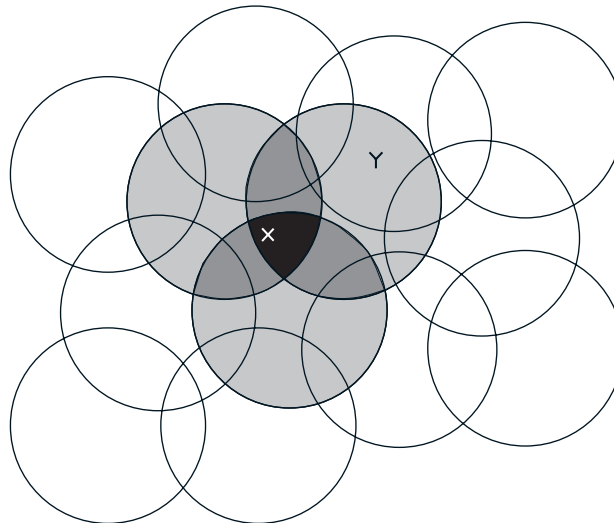


Coarse coding – Bynary features

If the state is INSIDE a circle → the corresponding feature has the *value 1*
(it is said to be *present*)

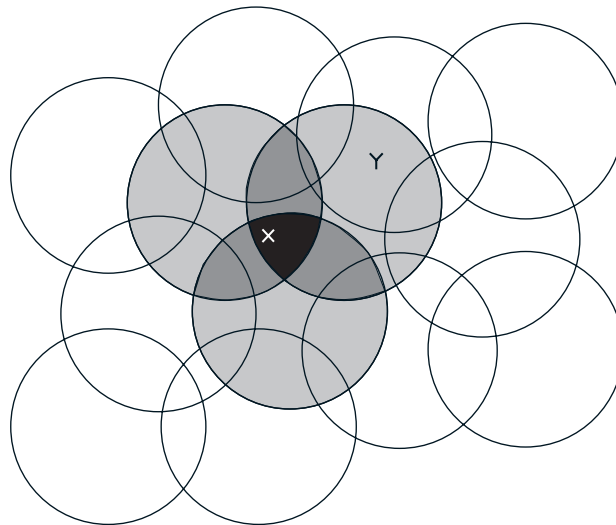
If the state is NOT INSIDE a circle → the corresponding feature has the
value 0 (it is said to be *absent*)

*Representing a state with features that overlap in this way (although they need not be circles or binary) is known as **COARSE CODING**.*



Coarse coding

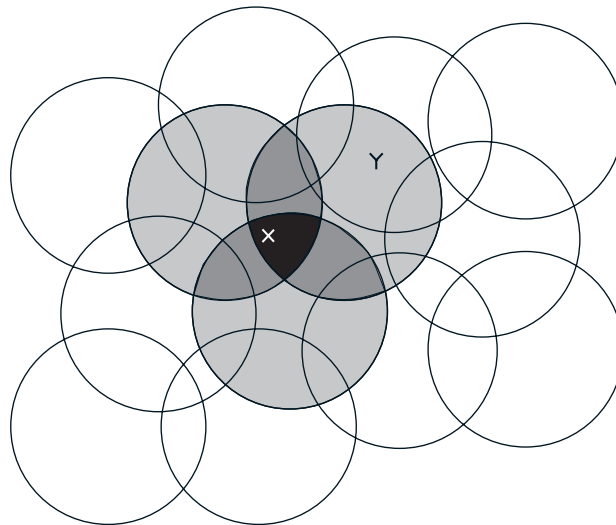
What are the effects of the size and density of circles on linear gradient-descent function approximation?



Coarse coding

*What are the effects of the **size and density of circles** on linear gradient-descent function approximation?*

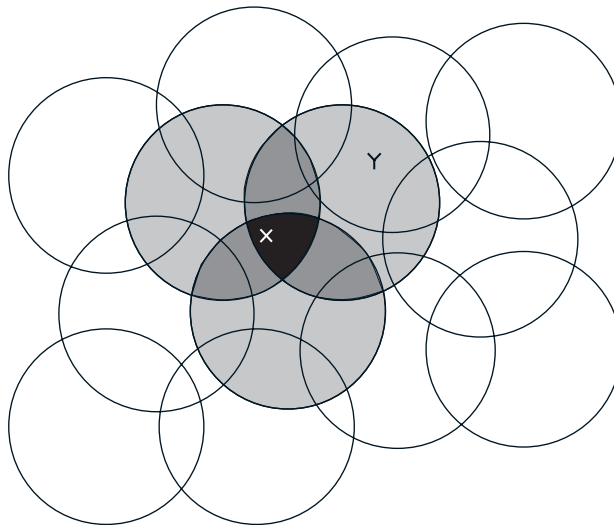
- Corresponding to each circle is a single parameter (a component of w) that is affected by learning. If we train at one point (state) X , then *the parameters of all circles intersecting X will be affected*.



Coarse coding

What are the effects of the size and density of circles on linear gradient-descent function approximation?

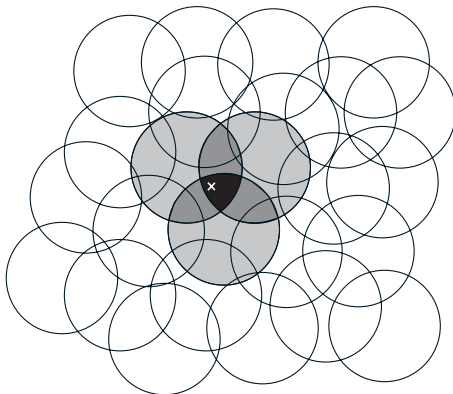
- Corresponding to each circle is a single parameter (a component of w) that is affected by learning. If we train at one point (state) X , then *the parameters of all circles intersecting X will be affected*.
- The approximate value function will be affected at *all points within the union of the circles*, with a greater effect the more circles a point has “in common” with X



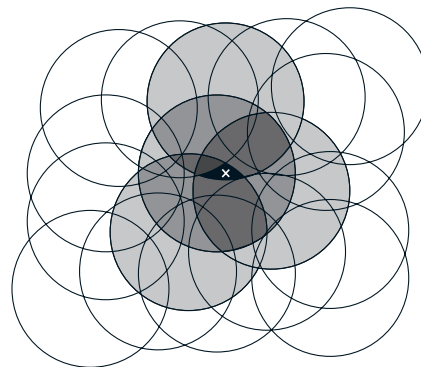
Coarse coding

If the *circles are small* → **generalization** will be over a *short distance*

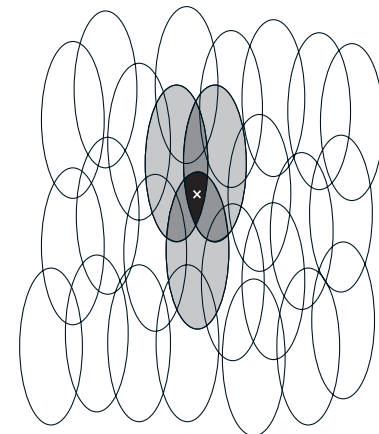
If the *circles are large* → **generalization** will be over a *large distance*



a) Narrow generalization



b) Broad generalization



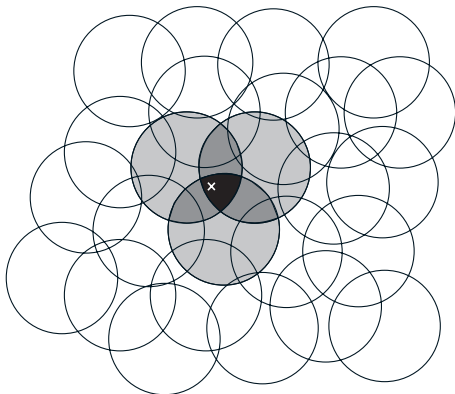
c) Asymmetric generalization

Coarse coding

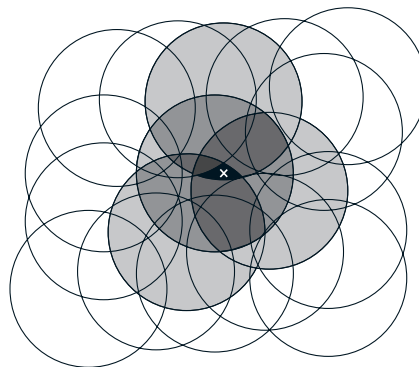
If the *circles are small* → **generalization** will be over a *short distance*

If the *circles are large* → **generalization** will be over a *large distance*

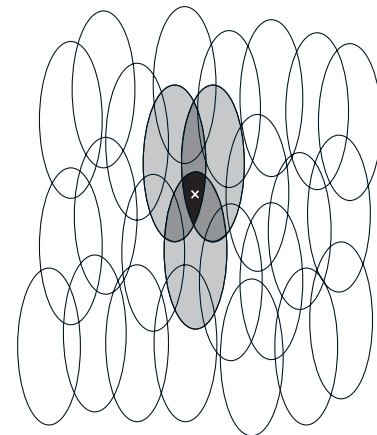
Observation : the *shape of the features* will determine the *nature of the generalization* (for example, if they are not strictly circular, but are elongated in one direction, then generalization will be similarly affected)



a) Narrow generalization



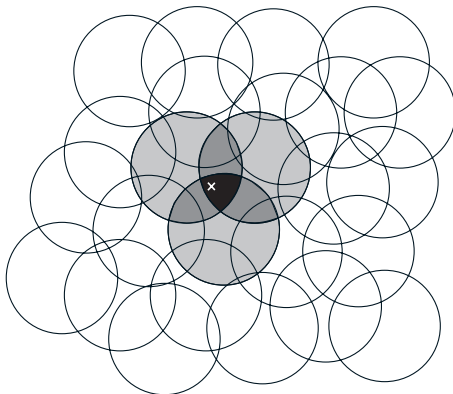
b) Broad generalization



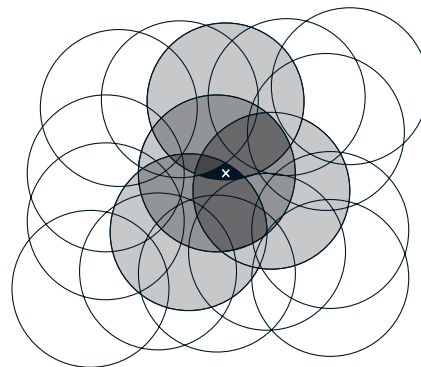
c) Asymmetric generalization

Coarse coding

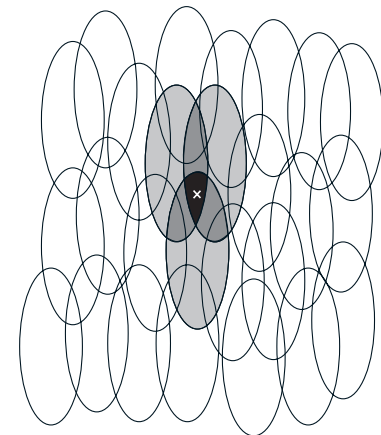
Intuition: Features with large receptive fields give broad generalization, but might also seem to limit the learned function to a coarse approximation (*unable to make discriminations much finer than the width of the receptive fields*)



a) Narrow generalization



b) Broad generalization



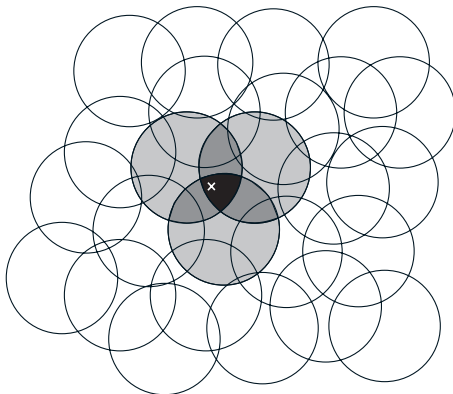
c) Asymmetric generalization

Coarse coding

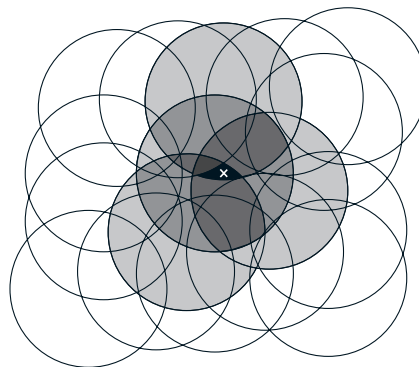
Intuition: Features with large receptive fields give broad generalization, but might also seem to limit the learned function to a coarse approximation (*unable to make discriminations much finer than the width of the receptive fields*)



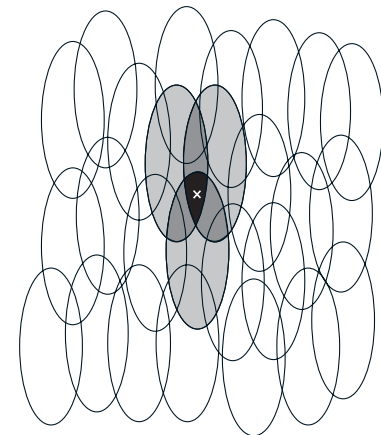
Not completely correct!



a) Narrow generalization



b) Broad generalization



c) Asymmetric generalization

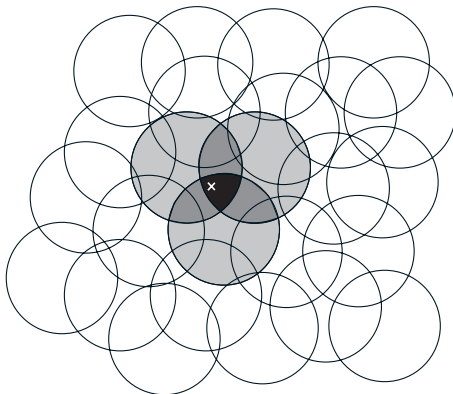
Coarse coding

Intuition: Features with **large receptive fields** give **broad generalization**, but might also seem to limit the learned function to a **coarse approximation** (*unable to make discriminations much finer than the width of the receptive fields*)

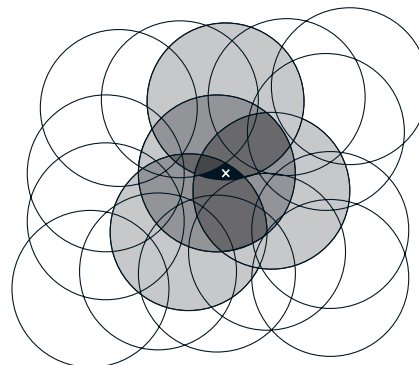
Not completely correct!

Two facts :

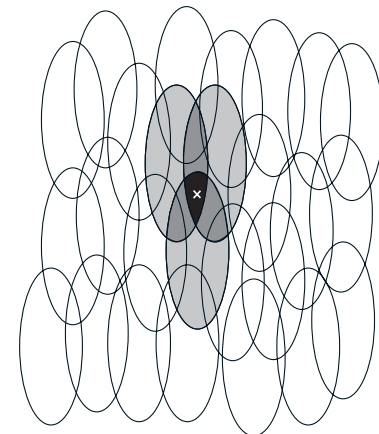
- *Initial generalization* from one point to another is indeed controlled by the *size and shape* of the receptive fields...
- ... but the *finest discrimination ultimately* possible, is controlled more by the *total number of features*.



a) Narrow generalization

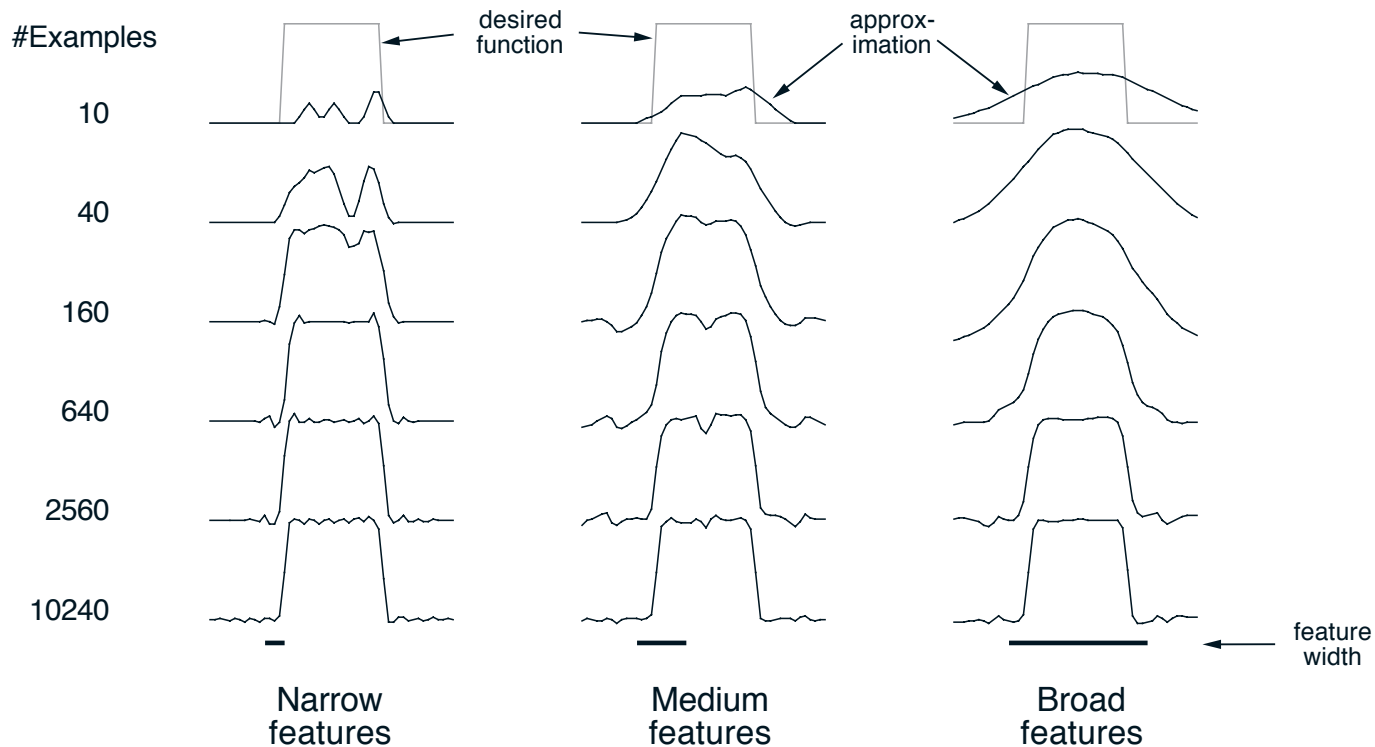


b) Broad generalization



c) Asymmetric generalization

Coarseness of Coarse coding



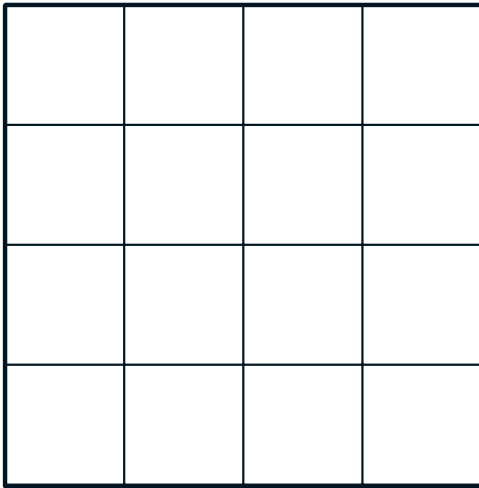
- The width of the features had a strong effect early in learning
- Final function learned was affected only slightly by the width of the features
- Receptive field shape tends to have a strong effect on generalization but little effect on asymptotic solution quality

Today's lecture

- How to build linear features (Coarse coding; **Tile coding**; Radial basis functions)
- Control with function approximation

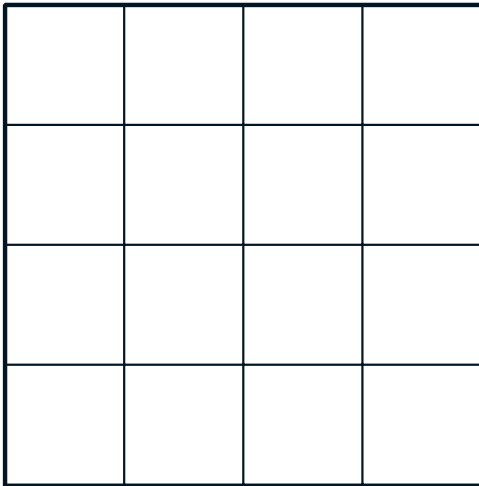
Tile coding

Suppose the receptive fields of the features are grouped into partitions of the state space



Tile coding

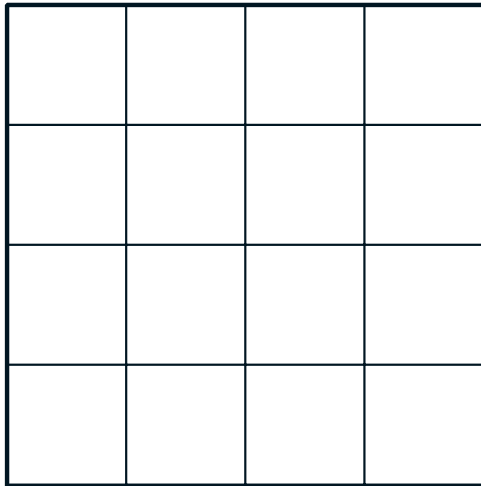
Suppose the receptive fields of the features are grouped into partitions of the state space



- The partition is called *tiling*
- Each element of the partition is called *tile*

Tile coding

Suppose the receptive fields of the features are grouped into partitions of the state space



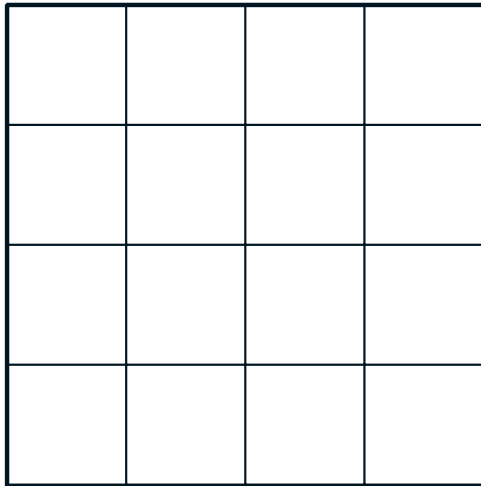
- The partition is called *tiling*
- Each element of the partition is called *tile*



In this two dimensional example, the partition is given by a uniform grid and the features are squares

Tile coding

Suppose the receptive fields of the features are grouped into partitions of the state space



- The partition is called *tiling*
- Each element of the partition is called *tile*

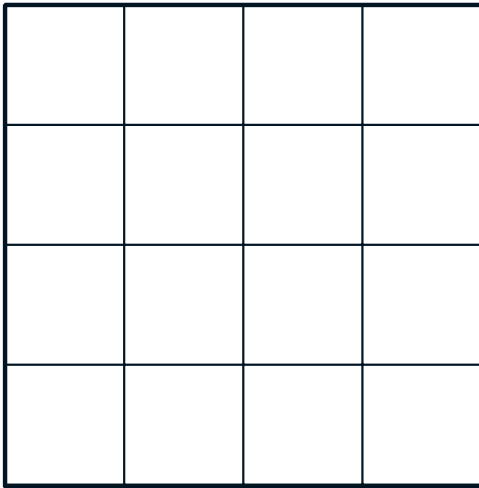


In this two dimensional example, the partition is given by a uniform grid and the features are squares

- The *state of a point* is represented by the *single feature* whose tile it falls within

Tile coding

Suppose the receptive fields of the features are grouped into partitions of the state space



- The partition is called *tiling*
- Each element of the partition is called *tile*



In this two dimensional example, the partition is given by a uniform grid and the features are squares

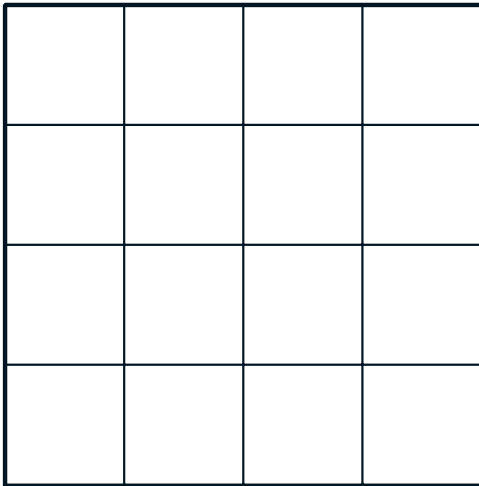
- The *state of a point* is represented by the *single feature* whose tile it falls within



Generalization would be complete to all states within the same tile and nonexistent to states outside it

Tile coding

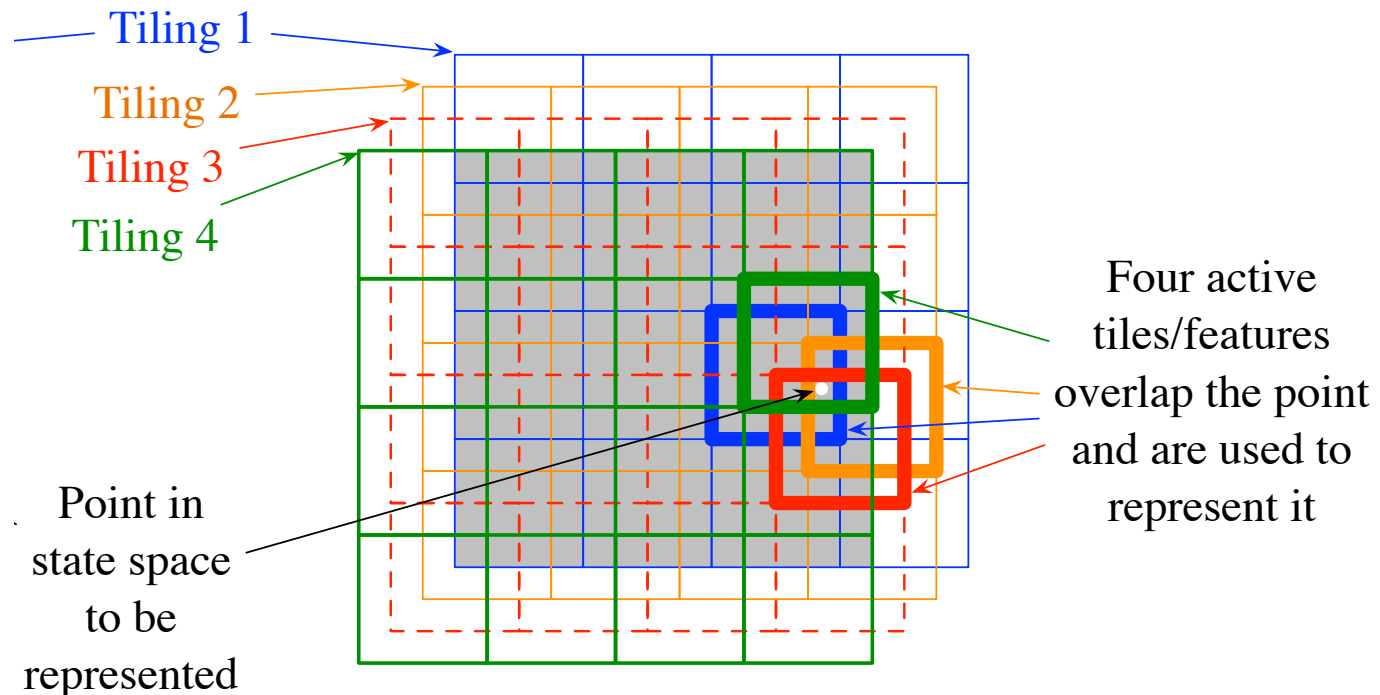
With just one tiling, we would not have coarse coding but just a case of state aggregation



Tile coding

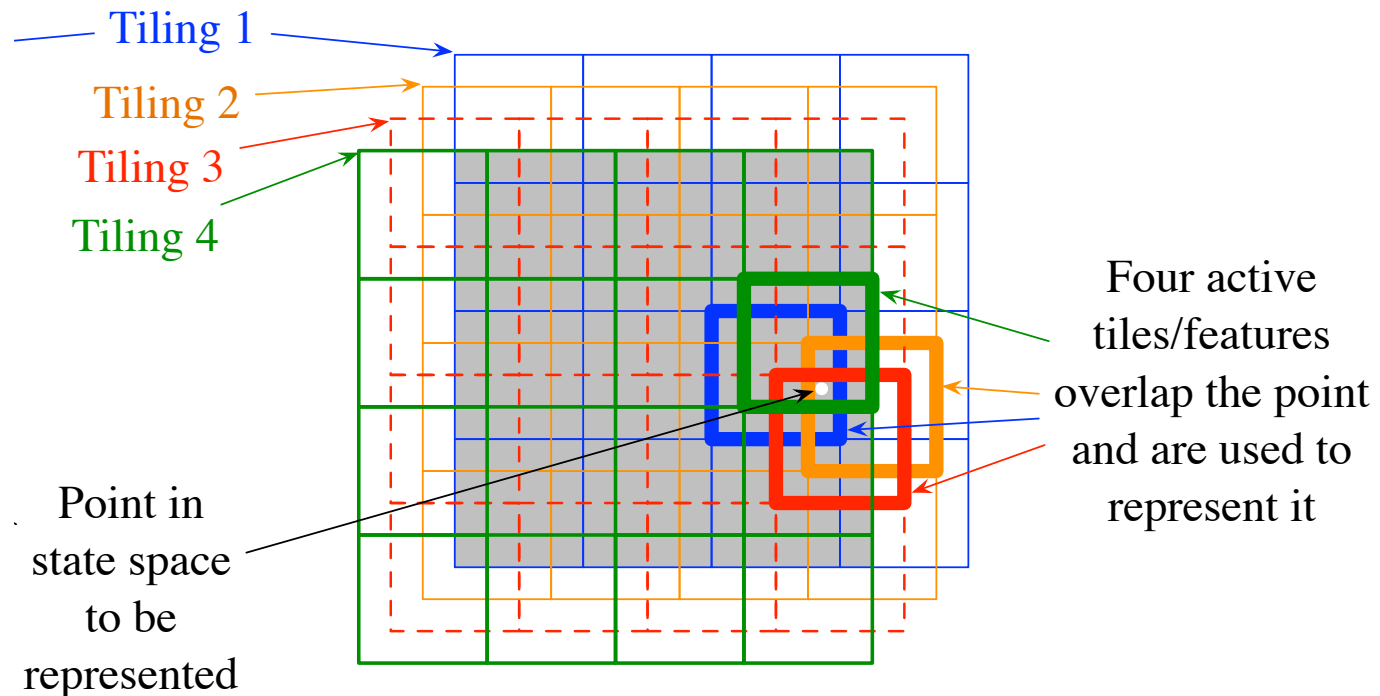
With just one tiling, we would not have coarse coding but just a case of state aggregation

To get true coarse coding with tile coding, *multiple tilings are used*, each offset by a fraction of a tile width



Tile coding

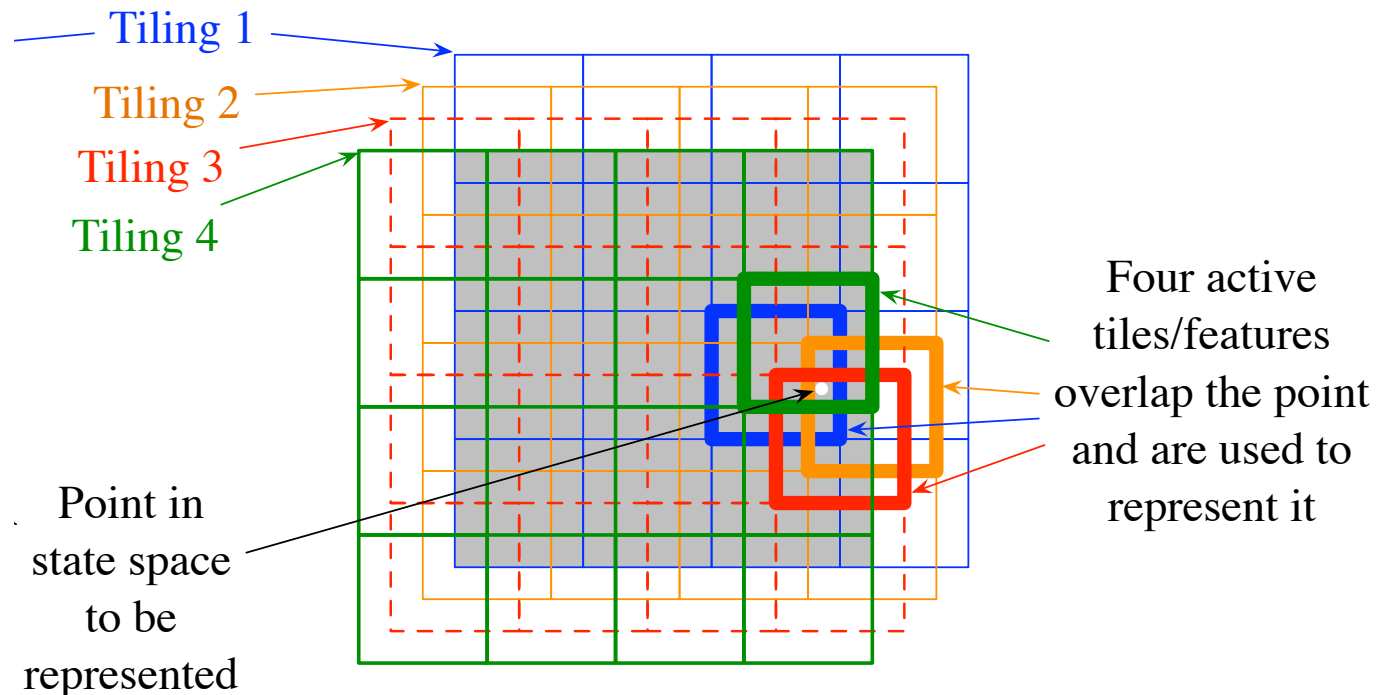
Every state, such as that indicated by the white spot, falls in exactly one tile in each of the four tilings.



Tile coding

Every state, such as that indicated by the white spot, falls in exactly one tile in each of the four tilings.

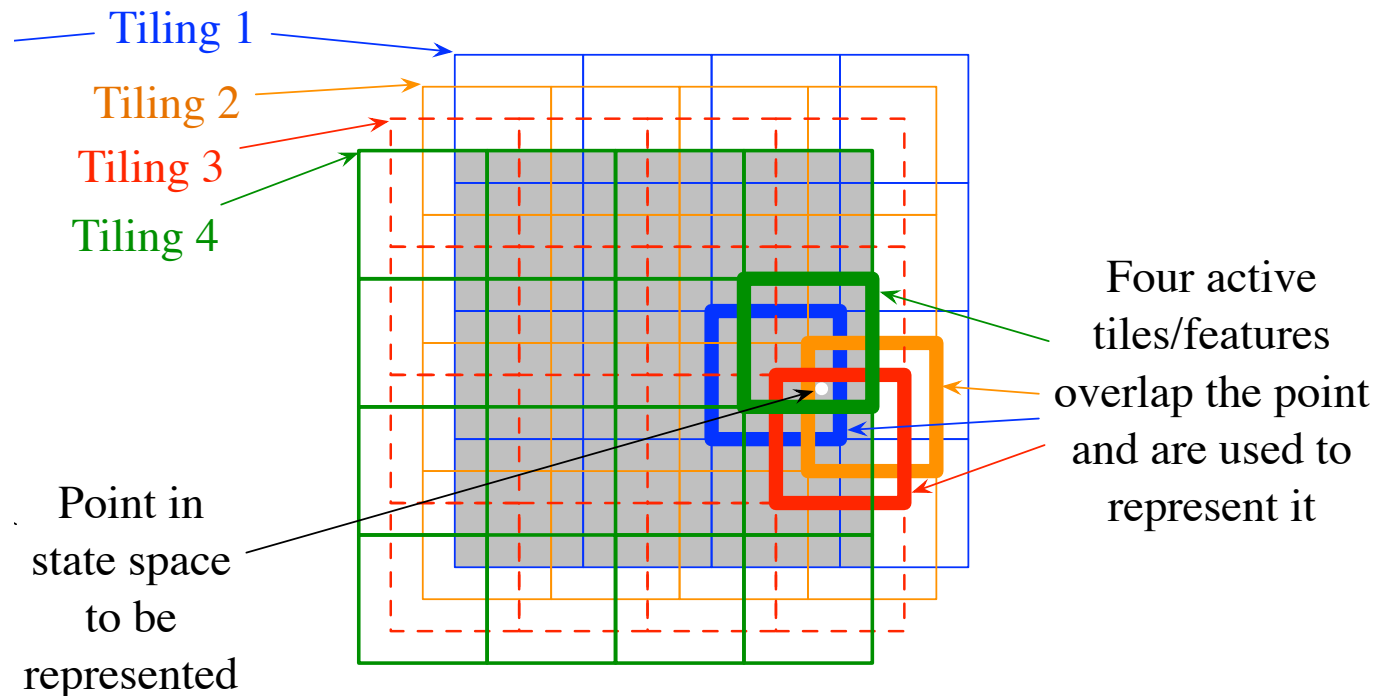
↳ *The four tiles correspond to four features that become active when the state occurs.*



Tile coding

Every state, such as that indicated by the white spot, falls in exactly one tile in each of the four tilings.

- The four tiles correspond to four features that become active when the state occurs.
- The feature vector $x(s)$ has one component for each tile in each tiling.

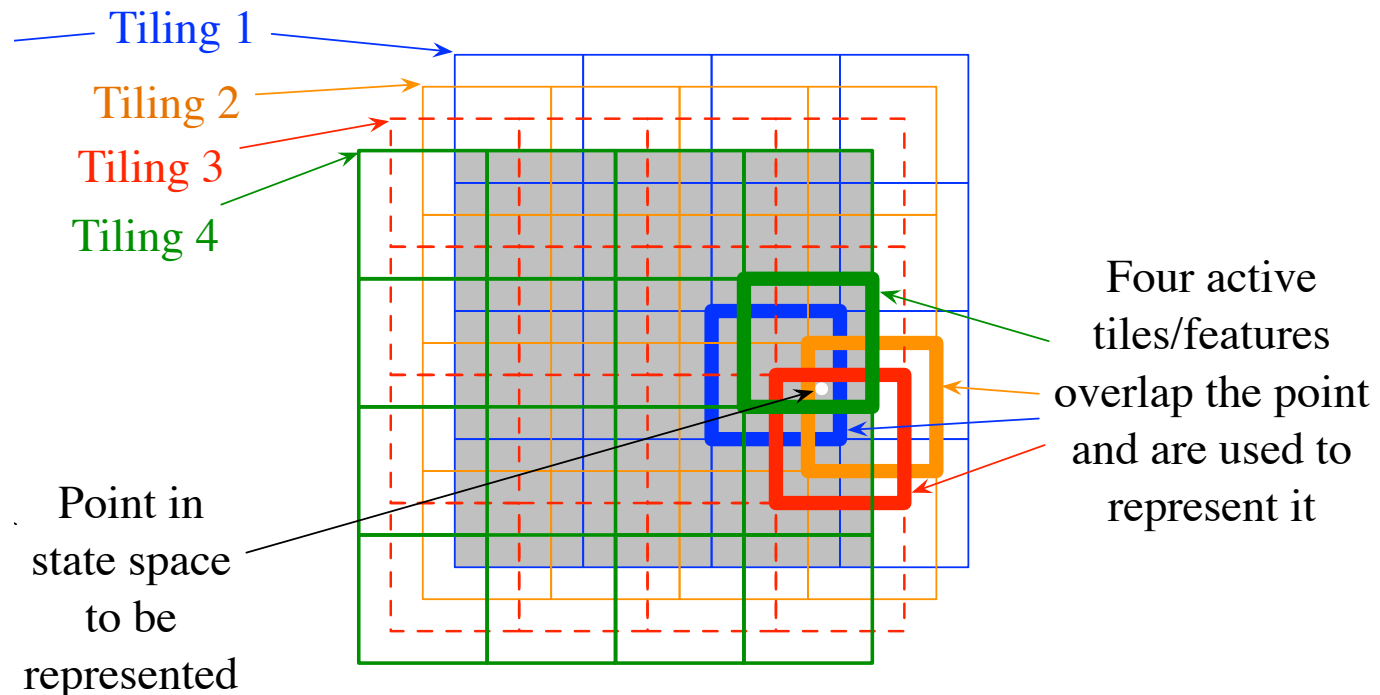


Tile coding

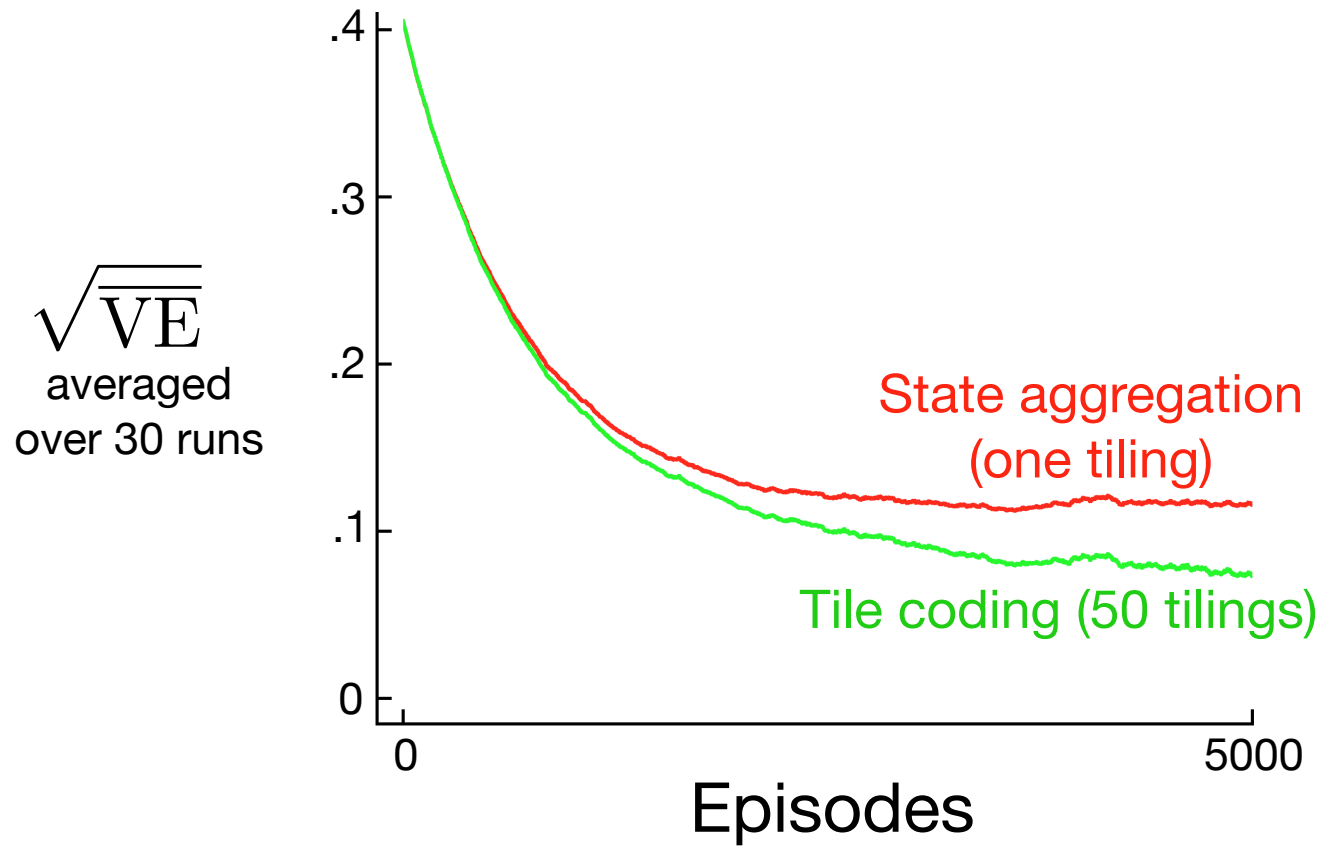
Every state, such as that indicated by the white spot, falls in exactly one tile in each of the four tilings.

- The four tiles correspond to four features that become active when the state occurs.
- The feature vector $x(s)$ has one component for each tile in each tiling.

In this example there are $4 \times 4 \times 4 = 64$ components, all of which will be 0 except for the four corresponding to the tiles that s falls within.



Example : 1000-state random walk



Tile coding : some considerations

- An immediate practical advantage of tile coding is that, because it works with partitions, *the overall number of features that are active at one time is the same for any state.*

Tile coding : some considerations

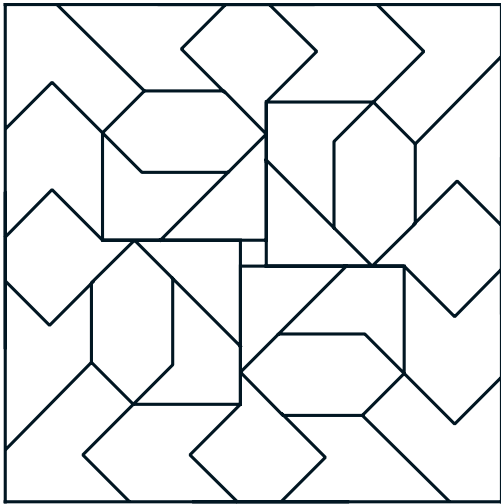
- An immediate practical advantage of tile coding is that, because it works with partitions, *the overall number of features that are active at one time is the same for any state.*
- Exactly one feature is present in each tiling, so the *total number of features present is always the same as the number of tilings.*

Tile coding : some considerations

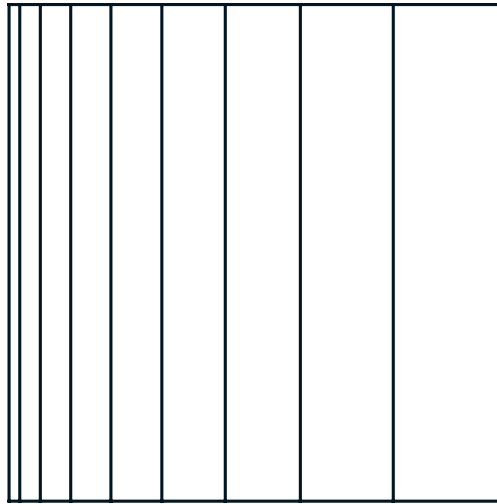
- An immediate practical advantage of tile coding is that, because it works with partitions, *the overall number of features that are active at one time is the same for any state.*
- Exactly one feature is present in each tiling, so the *total number of features present is always the same as the number of tilings.*
- Tile coding also gains *computational advantages from its use of binary feature vectors*. Indeed, because each component is either 0 or 1, the weighted sum making up the approximate value function is almost trivial to compute (*we will see an example in the Sarsa(λ)*)

Tile coding : some considerations

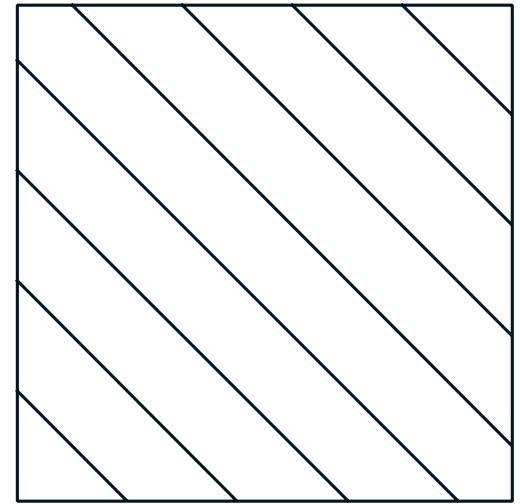
In choosing a tiling strategy, one has to pick the *number of the tilings* and the *shape of the tiles*.



Irregular



Log stripes

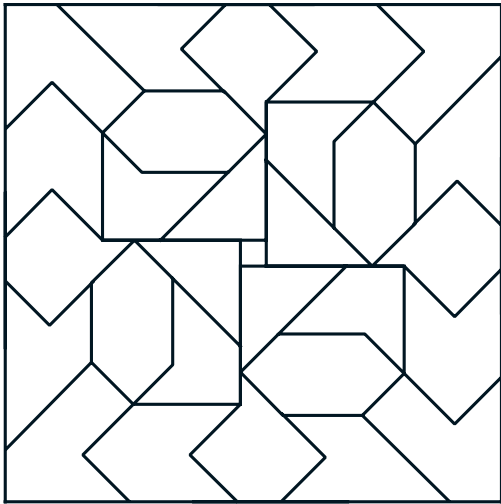


Diagonal stripes

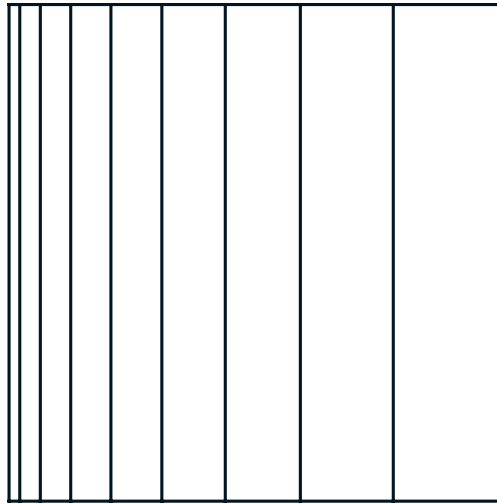
Tile coding : some considerations

In choosing a tiling strategy, one has to pick the *number of the tilings* and the *shape of the tiles*.

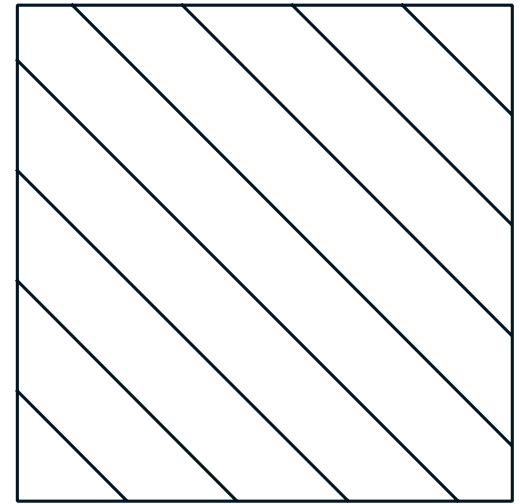
- The *number of tilings*, along with the *size of the tiles*, determines the *resolution* or fineness of the asymptotic approximation



Irregular



Log stripes

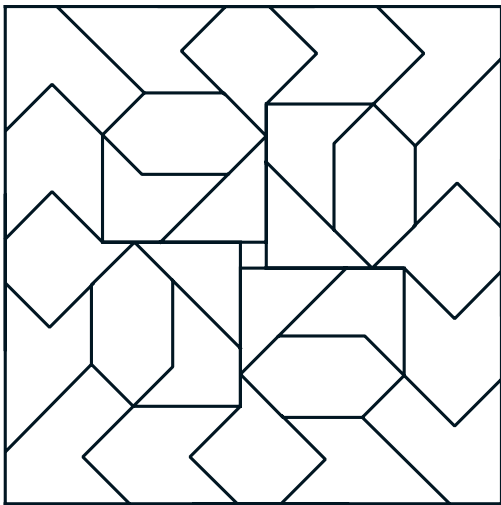


Diagonal stripes

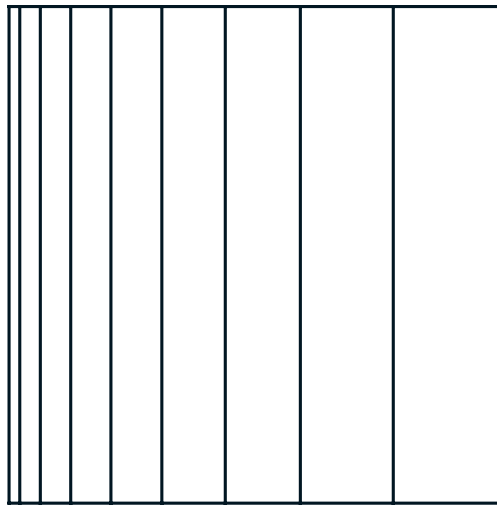
Tile coding : some considerations

In choosing a tiling strategy, one has to pick the *number of the tilings* and the *shape of the tiles*.

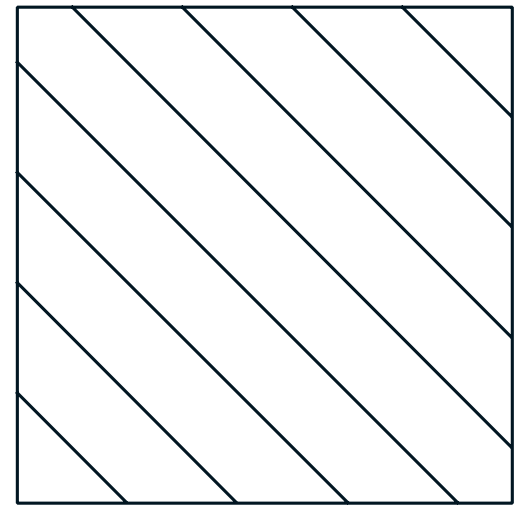
- The *number of tilings*, along with the *size of the tiles*, determines the *resolution* or fineness of the asymptotic approximation
- The *shape of the tiles* will determine the *nature of generalization*



Irregular



Log stripes



Diagonal stripes

Today's lecture

- How to build linear features (Coarse coding; Tile coding; **Radial basis functions**)
- Control with function approximation

Radial basis functions

Radial basis functions (RBFs) are the natural generalization of coarse coding to continuous-valued features.



each feature can be anything in the *interval* $[0, 1]$, reflecting various degrees to which the feature is present

Radial basis functions

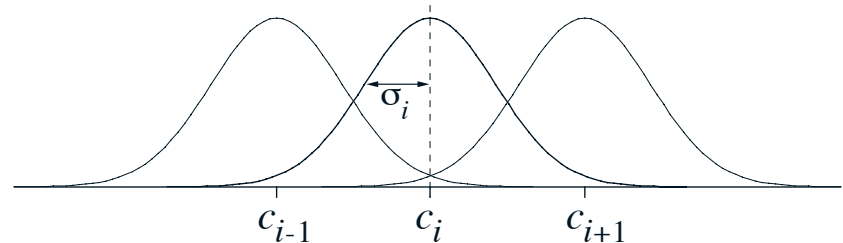
Radial basis functions (RBFs) are the natural generalization of coarse coding to continuous-valued features.



each feature can be anything in the *interval* $[0, 1]$, reflecting various degrees to which the feature is present

A typical RBF feature, x_i , has a Gaussian (bell-shaped) response $x_i(s)$ dependent only on the distance between the state, s , and the feature's prototypical or center state, c_i , and relative to the feature's width, σ_i :

$$x_i(s) \doteq \exp\left(-\frac{\|s - c_i\|^2}{2\sigma_i^2}\right)$$



Radial basis functions

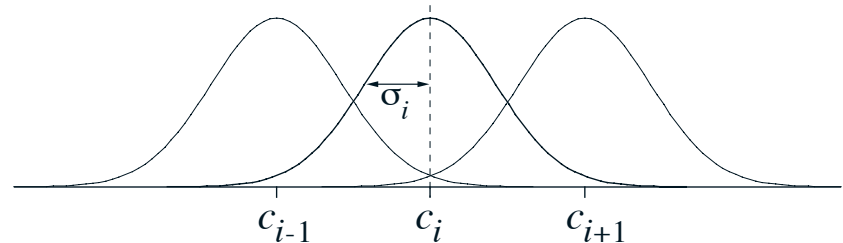
Radial basis functions (RBFs) are the natural generalization of coarse coding to continuous-valued features.



each feature can be anything in the *interval* $[0, 1]$, reflecting various degrees to which the feature is present

A typical RBF feature, x_i , has a Gaussian (bell-shaped) response $x_i(s)$ dependent only on the distance between the state, s , and the feature's prototypical or center state, c_i , and relative to the feature's width, σ_i :

$$x_i(s) \doteq \exp \left(-\frac{\|s - c_i\|^2}{2\sigma_i^2} \right)$$



The primary advantage of RBFs over binary features is that they produce approximate functions that vary smoothly and are differentiable

Material

The material illustrated so far can be found in paragraph 9.5
*(other types of features constructions for linear methods - polynomials
and Fourier basis)*